

The list index starts from zero, and an optional step can also be given. In the following example, the second element from the input list is returned.

```
(-slice list from to step) ;; Syntax
```

```
(-slice '(1 2 3 4 5) 2 3)
(3)
```

The ‘-take’ function returns a new list with the first N (number) elements from the input list. If N is greater than the number of elements in the list, then the entire list is returned. For example:

```
(-take N list) ;; Syntax
```

```
(-take 3 '(1 2 3 4 5))
(1 2 3)
```

The ‘-take-last’ function returns the last N items from the input list.

```
(-take-last N list) ;; Syntax
```

```
(-take-last 3 '(1 2 3 4 5))
(3 4 5)
```

The ‘-drop’ function removes the first N items from the input list, and returns the rest of the list (tail), while the ‘-drop-last’ function removes the last N items from the list and returns the rest of the list. A couple of examples are shown below:

```
(-drop N list) ;; Syntax
(-drop-last N list) ;; Syntax
```

```
(-drop 3 '(1 2 3 4 5))
(4 5)
```

```
(-drop-last 3 '(1 2 3 4 5))
(1 2)
```

The ‘-take-while’ function returns the input list until each element satisfies the predicate function. The function usage is demonstrated with the following examples:

```
(-take-while predicate list) ;; Syntax
```

```
(-take-while 'even? '(1 2 3))
nil
```

```
(-take-while 'even? '(2 3 4))
(2)
```

The ‘-drop-while’ function removes the elements from the input list as long as the predicate function matches, and returns the rest of the list. A couple of examples are shown below:

```
(-drop-while predicate list) ;; Syntax
```

```
(-drop-while 'even? '(1 2 3 4))
(1 2 3 4)
(-drop-while 'even? '(2 3 4 5))
(3 4 5)
```

The elements of a list at specific indices can be returned using the ‘-select-by-indices’ function. It takes a list of indices as an argument and an input list. For example:

```
(-select-by-indices indices list) ;; Syntax
```

```
(-select-by-indices '(0 2 3) '("a" "b" "c" "d" "e"))
("a" "c" "d")
```

A matrix or table can be represented as a list of lists. The elements in a specific column can be returned as a list using the ‘-select-column’ function. An example is given below:

```
(-select-column columns table) ;; Syntax
```

```
(-select-column 1 '((1 2 3) (4 5 6) (7 8 9)))
(2 5 8)
3.3.
```

List to list operations

The ‘-keep’ function takes a function as an argument and applies the former to each element in the list. It then returns a new list with those elements that return a non-nil value for the function application. For example:

```
(-keep function list) ;; Syntax
```

```
(-keep 'cdr '((7 8 9) (7 8) (7)))
((8 9) (8))
```

If you want to concatenate lists together, you can use the ‘-concat’ function as shown below:

```
(-concat '(1) '(2 3) '(4))
```

```
(1 2 3 4)
```

Nested lists can be flattened to a single list using the ‘-flatten’ function. You can also specify the level of the nested list to flatten using the ‘-flatten-n’ function. A couple of examples are provided below to illustrate the function’s use: